



university of
groningen

faculty of science
and engineering

Final Project Report

Introduction to Machine Learning (for AI)

WBAI056-05.2025-2026.1

Group Number:	23
Author(s):	Chiara Tenderini (S6511686) Ricardo Rubert Climent (S5909759) Tien Huy Anh Luu (S5906261)
Delivery Date:	January 31, 2026

1 Project Overview

In this project, we attempt to classify whether a user will watch a show using a KNN (K-Nearest-Neighbours) architecture. We train our model on a subset of the Netflix Prize Dataset. [1]

Given the Netflix Prize Dataset, we will select a user and their ratings for each movie. We use a database [2] to obtain information for each movie, after which we use a sentence transformer to embed the title and overview. We proceed to split our data into binary classes "Watch" and "Skip", indicating whether a user should/would engage with the movie. Consequently, we one-hot-encode categorical variables, perform a train-test split and normalize our features. At last, we train a KNN model using stratified k-fold cross validation to tune hyper-parameters. We provide a visualisation of the methodology in Figure 1.

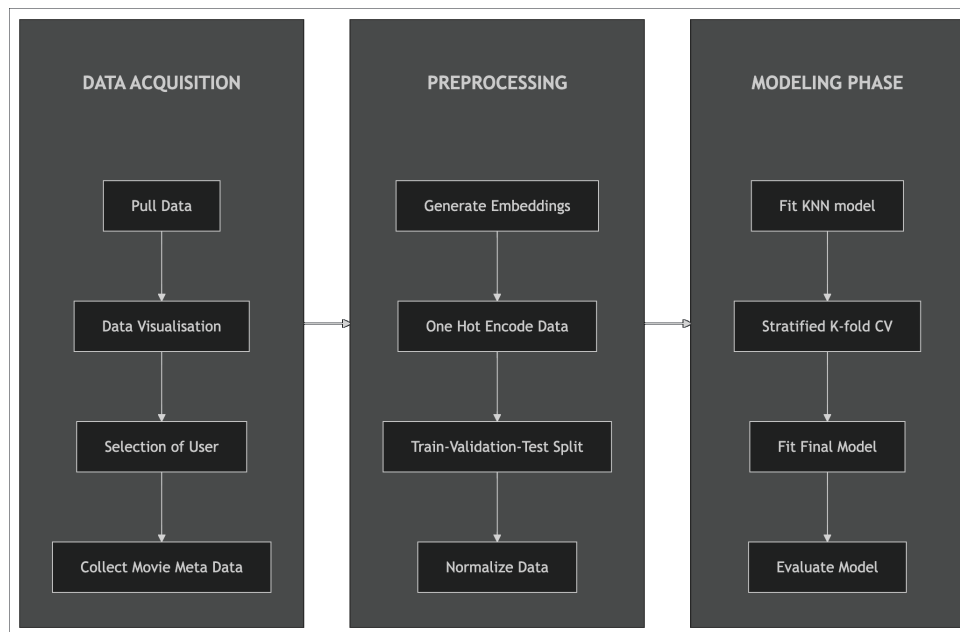


Figure 1: Workflow overview including the stages of data acquisition, preprocessing, modelling and their substeps

Our primary objective was to develop a model that achieves a classification performance significantly better than random chance. We chose this benchmark, rather than aiming for near-perfect accuracy, as an acknowledgment of the inherent imperfections of the dataset. Real-world recommendation data is often sparse and subject to 'noisy' labels; for instance, a user's tastes may shift over time in ways not capturable by historical data, it is also notable to say that the dataset had missing values that we had to account for. Given these constraints, our objective was to demonstrate that a KNN architecture can identify meaningful patterns and yield predictive power beyond a stochastic baseline.

This reports serves as a presentation of our findings. In the following sections, we shall provide: a visualisation of our data, justification for the employment of a KNN architecture, presentation of the results obtained, an analysis upon these results and at last a conclusion.

The complete source code and implementation details are available on GitHub at github.com/Cistaroth/IML-FINAL-PROJECT

2 Data

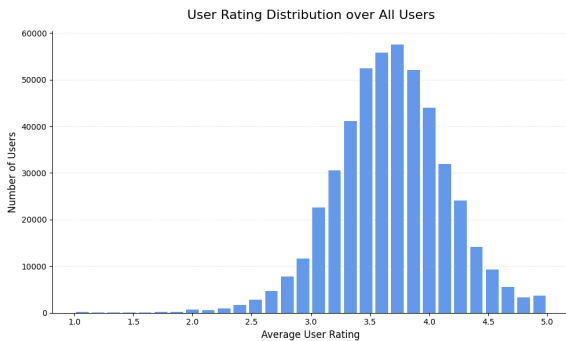
The complete dataset is available on Kaggle at: [kaggle.com/datasets/netflix-inc/netflix-prize-data](https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data)

2.1 Extraction from Raw Data

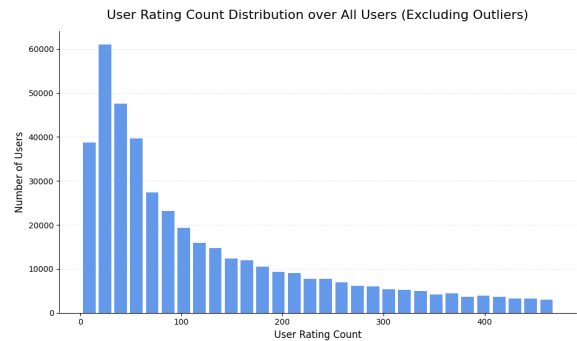
On October 2 in 2006, Netflix held an open competition for the best algorithm to predict user ratings for films with a grand prize of US\$1,000,000. The data set contains the ratings of 480189 users covering 1,777 movies. The dataset was contained by four files with the following structure for each file:

```
<movie-id>:
<user-id>, <Rating>, <Date>
...
<movie-id>:
<user-id>, <Rating>, <Date>
...
```

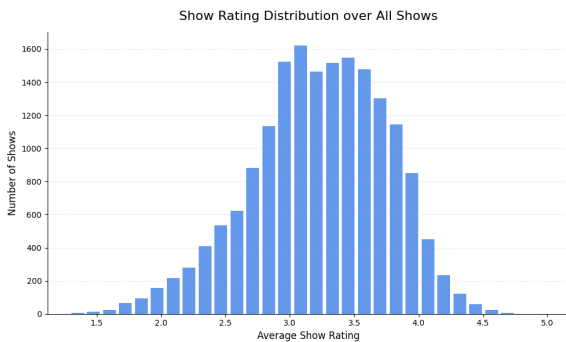
In the subsections that follow, we will outline how we adapted this dataset to adhere within the bounds of our project. The first step was to process the data into a data-frame for each rating with columns: `movie_id`, `user_id`, `rating`. Given the large size of the dataset, we run the risk of exceeding the RAM limit, as such manipulation of the data had to be performed with caution. We have summarized the distribution of mean rating and distribution of number of ratings across the entirety of the users and shows in Figure 2. We refer to Figure 3 for the distribution of ratings.



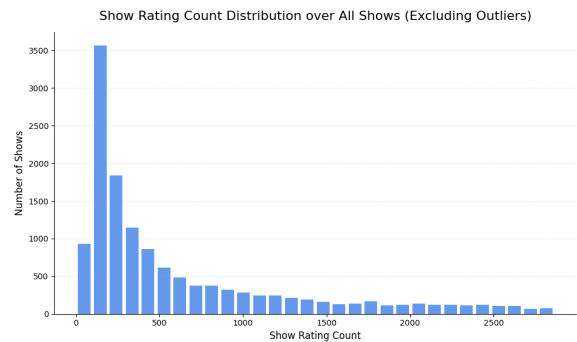
(a) Mean Rating across All Users



(b) Number of Ratings across All Users



(c) Mean Rating across All Shows



(d) Number of Ratings across All Shows

Figure 2: Visualisation of mean/count distributions over users/shows

We note that the average ratings across users and shows seems approximately normally distributed. The number of ratings is also normally distributed as shown in Figure 3.

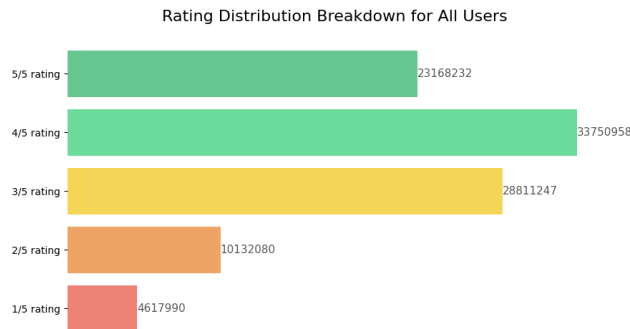


Figure 3: Distribution of Rating Distribution across dataset

Lastly, the dataset also contains the file "movies.json", which contains mappings from movie_ids to movie_titles. We use this mapping to transform our data-frame into a data-frame with columns: movie_title, user_id, rating.

2.2 Selecting user

After extracting the raw user data, we must select a specific user to perform classification on. We sought to select a user whom fits the following criteria: they must have an average rating of approximately three and they must have rated the greatest number of movies. Firstly, we seek an average rating of approximately three as this reduces class imbalances when we split the data into the classes "should watch" and "should not watch". Secondly, we seek to select the user with the most data as this helps reduce overfitting. Following these criteria, we find that the user with ID 1314869 is our best candidate out of 68330 potential candidates. Our selected user has a mean rating of 2.95 with standard deviation 0.98 across 9740 ratings.

2.3 Adding metadata

Given that the Netflix Prize Dataset is exclusive of movie information, we must use an external tool to obtain information pertaining to the movies. As such, we make API requests to an endpoint from The Movie DataBase [2] and attempt to obtain the following details: "overview", "year of release", "language", "media type", "vote average", "production countries", "genres", "credits", "runtime", "popularity", "rating". We would like to clarify that the popularity is an index generated by TMDb. In addition, the information on credits only contains the first five individuals in the credits of the show. After obtaining additional details, we concatenate the columns with additional information to the pre-existing data-frame. We could not find details for 1667 movies, accounting for approximately 17%. We find that the final data-frame has a mean rating of 2.97 with standard deviation 0.95 across 8076 ratings. We refer to Figure 4 for detailed breakdown of the ratings distribution.

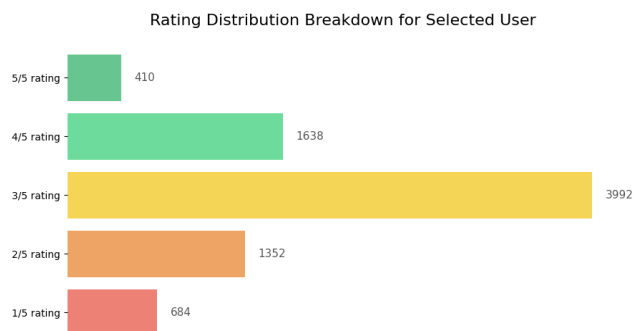


Figure 4: Rating Distribution of User 1314869

2.4 Generating Embeddings

In order to use title and overview, we use the sentence transformers provided by the sentence-transformers library [3]. We encode the title and overview using the "all-MiniLM-L6-v2" model. We chose this model due to its capability for clustering [4]. We may consider using a custom sentence transformer for movies; however, we could not find any models on hugging face. Moreover, we could consider fine-tuning a sentence transformer ourselves, however this would be beyond the scope of the course. For each column, title and overview, "all-MiniLM-L6-v2" generates 384 columns each to encode the information. Following generation of the embeddings, we remove the title and overview columns and replace the columns with our embeddings. Finally, we obtain our resultant dataframe consisting of 784 columns and 8076 rows.

2.5 Splitting into Skip and Watch classes

Next, we transform the problem into a binary classification problem by splitting the movies into two classes "Skip" and "Watch". The former indicates movies that should/would not be watched, while the latter indicates that a movie should/would be watched. We split based on the median rating in the training dataset. If movies have a lower rating than the median rating in the training set, they are assigned to the "Skip" class. If movies have a higher rating than the median rating training set, they are assigned to the "Watch" class.

3 Model

The model we will employ is a K-Nearest Neighbour model (KNN) for classification. We use this model as we can exploit the embeddings, where descriptions with similar semantics should have vectors closer in the embeddings space.

3.1 Code implementation

For the actual code, we utilised the library scikit-learn [5]. There are a few notable design choices that should be explained.

Firstly, we know that KNN suffers greatly from the curse of dimensionality, our data has a total of 778 columns where most of these are utilized for our model. This is a extremely large amount of

dimensions for KNN, so we looked at the different dimensionality reduction methods that we studied during the course to combat this. Sadly, the three dimensionality reduction methods taught during the course do not apply nicely to our data.

- Principal Component Analysis (PCA) - Needs the data to be linear in order to reduce the dimensions, our data exhibits non-linear relationships.
- t-Stochastic Neighbour Embedding (t-SNE) and Multi-Dimensional Scaling - Should only be used for data visualisation, not for actually putting the data through the pipeline.

Finally, it should also be stated that we assumed a very small amount of vocabulary leakage to increase the simplicity of our code. The leakage can be found when one hot encoding the cast from the movie. When we one hot encode it, we take the top 20 most popular actors of the entire data set and trim the rest. This adds a little bit of leakage because the model will now know who the famous actors are overall. However, if we would run into dimensional conflicts during our validation and our test that would need to be solved, adding an incredible amount of complexity to our code. Therefore, we are assuming some minor vocabulary leakage for the simplicity of our code.

3.2 Data split

We split our data into 80% training, 20% test. Then, during the hyper-parameter tuning, we utilize a Stratified K Fold with a K value of 5 to validate our results.

3.3 Normalisation

KNN works based on distance, so if a movie has a very long runtime, then its going to find itself much further away from all other movies and if a movie will be liked or not will depend on large metrics like runtime. Therefore, we need a normalisation technique. We utilized MinMaxScaler from the library scikit-learn [5] as our normalization technique to achieve normalised data.

3.4 Dealing with missing values

To deal with the missing values we utilized the SingleImputer from scikit-learn [5] and utilized the median of the value. For example, if we are missing the runtime from a movie, we replaced the NaN value with the value of the median of the training data set.

4 Results

4.1 Hyperparameter turning

We tuned the number of Neighbors K by evaluating balanced accuracy on the validation set across a range of values (more about why we utilized balanced accuracy in 5.1). As shown in the figure below, performance peaks at $K = 9$, achieving a validation balanced accuracy of approximately 66%. For large values of K, validation balanced accuracy gradually decreases, indicating that larger neighbourhoods smooth the decision boundary and increasing bias predictions toward the majority class. It is also interesting to see that the even values of K have a significantly lower balanced accuracy due to how KNN inherently works.

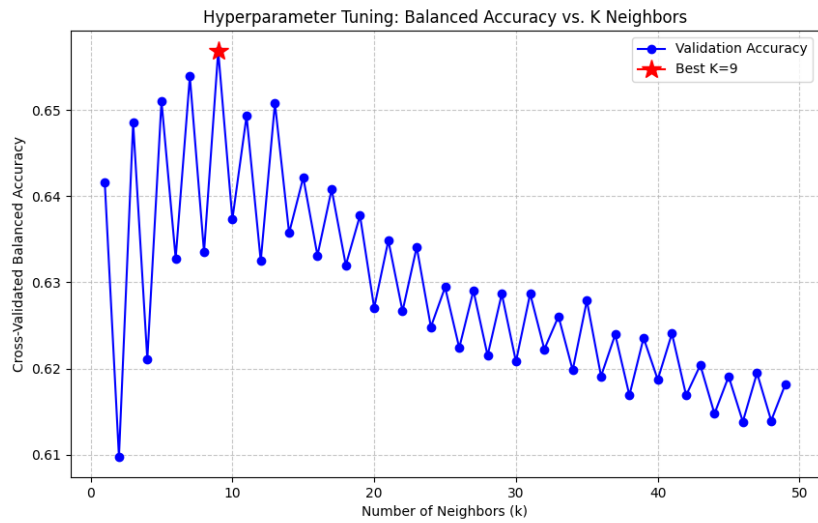


Figure 5: Hyperparameter tuning: Balanced accuracy vs number of neighbors (K). Best $K = 9$

4.2 Final model performance on the test set

Using $K = 9$, our final evaluation on the test set Figure 5 shows an overall accuracy of 0.80, with a balanced accuracy of 0.68 and a ROC-AUC of 0.75 (all results can be found in Table 1 and in Figure 7).

While the accuracy suggests strong performance, the balanced accuracy provides a more informative view of how well the model performs across both classes. In particular, the results reveal a clear asymmetry between predicting Skip and predicting Watch.

For the Skip class, the model performs very well: it achieves precision of 0.83, recall of 0.93 and an F1 score of 0.87 over 1210 test examples. In contrast, performance of the Watch class is weaker. Although the model's precision is 0.66, its recall is only 0.42, indicating that it fails to identify a large portion of the titles the user would actually watch.

This lower recall leads to a more modest Watch F1 score of 0.52 across 406 Watch examples. Overall, these results suggest that the model is reliable at detecting titles to skip, but tends to be conservative in recommending titles to watch.

4.3 Confusion matrix

To better understand where the model succeeds and fails, we analyse the confusion matrix Figure 6.

Out of 1210 true Skip items, the model correctly predicts 1123 as Skip and incorrectly recommends 87 as Watch. This means the model rarely recommends something that should be skipped. However, this situation is different for the Watch class. Out of 406 true Watch items, the model correctly identifies 172 as Watch but predicts 234 as Skip. These false negatives explain the low recall for the Watch class: the model frequently fails to surface titles that the user would actually enjoy.

Table 1: Performance Metrics of the KNN Model (Optimal $K = 9$)

Class	Precision	Recall	F1-Score	Support
Skip	0.83	0.93	0.87	1210
Watch	0.66	0.42	0.52	406
Accuracy			0.80	1616
Macro Avg	0.75	0.68	0.70	1616
Weighted Avg	0.79	0.80	0.79	1616
<i>Additional Metrics:</i>				
Balanced Accuracy: 0.68		ROC-AUC Score: 0.75		

Although this reflects relatively low recall, in a recommendation setting this conservative behaviour can be preferable: it is often better to miss some good titles (false negatives) than to recommend titles the user would likely dislike (false positives).

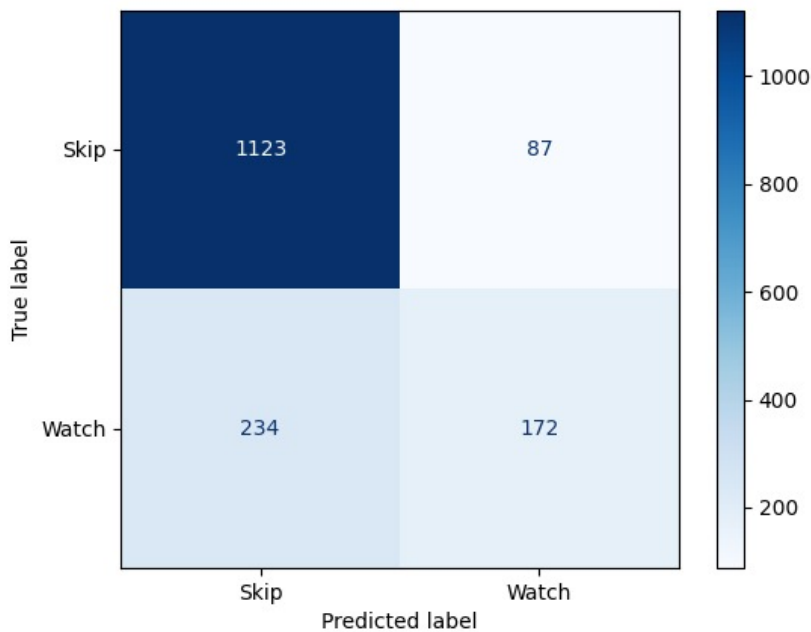


Figure 6: Confusion matrix on the test set

4.4 ROC curve

Finally, we evaluate discrimination performance using the ROC curve Figure 7. The model achieves a ROC-AUC of 0.75, which is substantially above the 0.50 level expected from random guessing. This shows that the model has meaningful predictive signal and it tends to rank Watch items above Skip items more often than chance. This is particularly relevant for recommendation, where the model can be used to generate a ranked list and only show the highest-confidence recommendations.

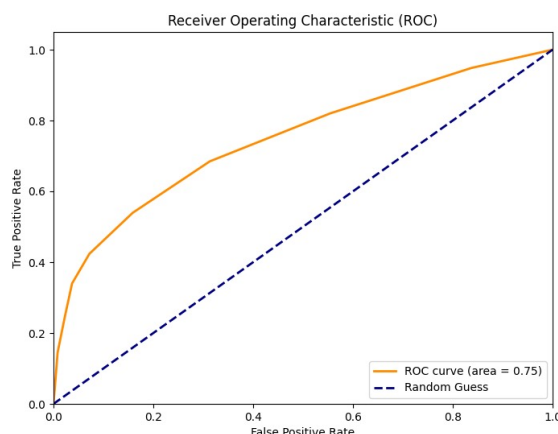


Figure 7: Figure 4. ROC curve for the Watch class (AUC = 0.75).

5 Analysis

5.1 Why accuracy is not a sufficient metric

Although the test accuracy is 0.80, accuracy alone is not a reliable metric here because the test set is imbalanced, with 1210 Skip examples and 406 Watch examples. In this setting, a trivial classifier that always predicts the majority class (Skip) already achieves an accuracy of: $1210/1616 = 0.749$.

So the improvement in accuracy from 0.75 to 0.80 is real but not as impressive as it initially appears. This is exactly why we emphasize balanced accuracy, which accounts for performance on both classes. The majority-class baseline would have a balanced accuracy of 0.50, whereas our KNN model reaches a balanced accuracy of 0.68, showing that it captures meaningful signal beyond simply exploiting the class distribution.

5.2 False negative

The main weakness of our model is the large number of false negatives for the Watch class. In other words, many items that should be recommended are instead predicted as Skip.

In practice, the model behaves like a careful recommender. It doesn't recommend many "bad" titles (only 87 false positives) and it's very good at spotting what the user would skip (Skip recall 0.93) Figure 6. The trade-off is that it's sometimes too careful: it also leaves out a lot of titles the user would actually like, which hurts discovery. Whether that's acceptable depends on personal choice; if you'd rather avoid annoying the user with poor suggestions, this is fine; if you want to surface as many good options as possible, then recall needs to improve at the cost of showing non-interesting titles.

5.3 Why K=9 is a reasonable choice

Figure 5 shows a peak at $K = 9$, where the validation balanced accuracy reaches around 65.69%. Beyond this point, performance generally declines as K increases. This trend is consistent with typical

KNN behaviour. When K is small, the model relies on very local neighbourhoods; a few neighbours can strongly influence the prediction, making the classifier more sensitive to noise and leading to higher variance. As K grows, predictions become more stable, but the model also starts averaging over many less-relevant neighbours, increasing the bias.

Therefore, our model choosing $K = 9$ represents a good compromise between capturing local patterns and maintaining stable predictions.

6 Conclusion

In conclusion, we achieved our goal of implementing a KNN (K-Nearest Neighbour) model capable of classifying a movie by leveraging embeddings and metadata with a performance significantly better than random chance. The complete source code and implementation details are available in the attachments and on GitHub at github.com/Cistaroth/IML-FINAL-PROJECT.

We started by acquiring our data, selecting a user, adding metadata, and generated embeddings. Consequently, we created a pipeline that can deal with missing values in our data and effectively found a way to normalise our data. Lastly, we were able to undergo a robust hyper-parameter search and tune our model to find the optimal K -value. We would like to refer to Figure 1 for a detailed breakdown.

We had to overcome many challenges, including: utilizing sentence transformers for the first time, managing high-dimensional categorical data, and lowering the bias in our highly imbalanced dataset (predominantly of the "Skip" class). Despite these challenges, we managed to achieve a final 68% balanced accuracy on our test set.

While we consider this project a success, the "curse of dimensionality" remains a limitation of the KNN architecture. Further work could improve performance by exploring dimensionality reduction techniques or alternative architecture which are less sensitive to high dimensions. Given that we could not find metadata for 17% of the original user data, we also suggest exploring other databases in future works. For further work, we would also consider using a different architecture which may exploit finding similar users for classification; for which exploration was not possible due to the constraints of this project. Despite these constraints, we believe that the model serves as a strong proof of concept for embedding-based movie classification.

7 Statement on Project Contribution

Chiara - Chiara analyzed the results, interpreted the figures, and wrote the Results and Analysis sections of the report.

Ricardo - Ricardo implemented the end-to-end machine learning pipeline using embeddings prepared by Huy Anh. These include data curation (One-Hot Encoding, normalization), developing the KNN model logic, executing hyper-parameter tuning, and generating the final evaluation metrics and figures.

Huy Anh - Huy Anh implemented the data acquisition, visualisation and generation of embeddings. Additionally, Huy Anh drafted the project overview and data section of the report.

Bibliography

- [1] Netflix, Inc., “Netflix prize data.” <https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data>, 2026. Accessed: 2026-01-31.
- [2] TiVo Platform Technologies, “The movie database (tmdb) api documentation.” <https://developer.themoviedb.org/docs/getting-started>, 2026. Accessed: 2026-01-31.
- [3] N. Reimers and H. Face, “Sentence transformers documentation.” <https://www.sbert.net/>, 2026. Accessed: 2026-01-31.
- [4] N. Reimers and H. Face, “all-minilm-l6-v2: A sentence-transformers model.” <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2021. Accessed: 2026-01-31.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.